Exp. No.	List of Experiments	Page No.
1.	Conditional Statements a. If b. If..Else c. If..Elif..Else d. Switch	2-6
2.	Control Statement a. For Loop b. While Loop c. Repeat Loop	7-10
3.	Function a. User Defined Function b. Recursive Function	11-13
4.a	Vector	14
4.b	List	17
5.	Array	18
6.	Data Interface: Excel File, Csv Files	21
7.	Graphs: Univariate, Bivariate And Multivariate	25
8.	Plot Function: Correlation Plots, Survival Plots and Mosaic Plots	28
9.	Measure Of Central Tendency: Mean, Median And Mode	31
10.	Standard Deviation	32
11.	Correlations And Covariance	33
12.	Linear Regression a. Simple Linear Regression b. Multiple Linear Regression	34-37

EX.NO:1a	CONDITIONAL STATEMENT- IF

Aim:

Write R Program to Check if a person is eligible to vote based on their age using if statement.

Program:

```
age <- as.integer(readline(prompt = "Enter your age: "))
if (age >= 18) {
  print("You are eligible to vote.")
}else{
  print("You are not eligible to vote.")
}
```

Output:

Enter your age: 25

"You are eligible to vote."

Result:

Thus the R program to check if a person is eligible to vote based on their age using if statement has been executed successfully.

EX.NO:1b	CONDITIONAL STATEMENT- IF -ELSE

Aim:

Write R Program to Check if a character entered by the user is a vowel or consonant using if else statement.

Program:

```
char <- tolower(readline(prompt = "Enter a character: "))
if (char %in% c("a", "e", "i", "o", "u"))
{
  print("The character is a vowel.")
}else{
  print("The character is a consonant.")
}
```

Output:

```
Enter a character: e
"The character is a vowel."
```

Result:

Thus the R program to check if a character entered by the user is a vowel or consonant using if else statement has been executed successfully.

EX.NO:1c **CONDITIONAL STATEMENT- IF –ELSE IF**

Aim:

Write R Program to Find the Largest of Three Numbers using if else if statement

Program:

```
num1 <- as.integer(readline(prompt = "Enter the first number: "))
num2 <- as.integer(readline(prompt = "Enter the second number: "))
num3 <- as.integer(readline(prompt = "Enter the third number: "))
if (num1 >= num2 && num1 >= num3)
{
  print(paste(num1, "is the largest number."))
}else if (num2 >= num1 && num2 >= num3)
{
  print(paste(num2, "is the largest number."))
}else{
  print(paste(num3, "is the largest number."))
}
```

Output

```
Enter the first number: 25
Enter the second number: 40
Enter the third number: 14
"40 is the largest number."
```

Result:

Thus the R program to Find the Largest of Three Numbers using if else if statement has been executed successfully.

EX.NO:1d	CONDITIONAL STATEMENT- IF

Aim:

Write R Program to perform addition subtraction, Multiplication and Division using Switch Case

Program:

```
add <- function(x, y)
{
return(x + y)
}
subtract <- function(x, y)
{
return(x - y)
}
multiply <- function(x, y)
{
return(x * y)
}
divide <- function(x, y)
{
return(x / y)
}
print("Select operation.")
print("1.Add")
print("2.Subtract")
print("3.Multiply")
print("4.Divide")
choice = as.integer(readline(prompt="Enter choice[1/2/3/4]: "))
num1 = as.integer(readline(prompt="Enter first number: "))
num2 = as.integer(readline(prompt="Enter second number: "))
```

```
operator <- switch(choice,"+","-","*","/")
result <- switch(choice, add(num1, num2), subtract(num1, num2),
multiply(num1, num2), divide(num1, num2))
print(paste(num1, operator, num2, "=", result))
```

Output:

```
[1] "Select operation."
[1] "1.Add"
[1] "2.Subtract"
[1] "3.Multiply"
[1] "4.Divide"
Enter choice[1/2/3/4]: 1
Enter first number: 15
Enter second number: 40
[1] "15 + 40 = 55"
```

Result:

Thus the R program to perform addition subtraction, Multiplication and Division using Switch Case has been executed successfully.

EX.NO:2 a	CONTROL STATEMENT - FOR LOOP

Aim:

Write R Program to Generating Prime Numbers using for loop

Program:

```
n <- as.integer(readline(prompt = "Enter a number: "))
cat("Prime numbers between 1 and", n, "are:\n")
for (num in 2:n)
 {
  is_prime <- TRUE
  for (i in 2:(num - 1))
 {
    if (num %% i == 0)
 {
      is_prime <- FALSE
      break
    }
  }
  if (is_prime)
 {
    print(num)
  }
 }
```

Output:

Enter a number: 15

Prime numbers between 1 and 15 are:

[1] 3

[1] 5

[1] 7

[1] 11

[1] 13

Result:

Thus the R program to Generating Prime Numbers using for loop has been executed successfully.

EX.NO:2 b	CONTROL STATEMENT - WHILE LOOP

Aim:

Write R Program to Finding sum of series given numbers using while loop

Program:

```
num = as.integer(readline(prompt = "Enter a number: "))
 if(num < 0)
{
 print("Enter a positive number")
 } else {
 sum = 0
while(num > 0)
{
 sum = sum + num
 num = num - 1
 }
 print(paste("The sum is", sum))
 }
```

Output:

```
Enter a number: 10
[1] "The sum is 55"
```

Result:

Thus the R program to Finding sum of series given numbers using while loop has been executed successfully.

EX.NO:2 c	CONTROL STATEMENT - REPEAT LOOP

Aim:

Write R Program to Generating Fibonacci Series using Repeat loop

Program:

```
n <- as.integer(readline(prompt = "Enter the number of terms: "))
a <- 0
b <- 1
cat("Fibonacci series: ", a, b, sep = "  ")
i <- 3
repeat
{
  c <- a + b
  cat(c, sep = "  ")
  a <- b
  b <- c
  i <- i + 1
  if (i > n) break
}
cat("\n")
```

Output:

Enter the number of terms: 10

Fibonacci series: 0 1 1 2 3 5 8 13 21 34

Result:

Thus the R Program to Generating Fibonacci Series using Repeat loop has been executed successfully.

EX.NO:3a	USER DEFINED FUNCTIONS

Aim:

To write R Program to find the area of different shape using user defined functions.

Program:

```
area_circle <- function(radius)
{
  pi * radius^2
}
area_rectangle <- function(length, width)
{
  length * width
}
area_triangle <- function(base, height)
{
  0.5 * base * height
}
 calculate_area <- function()
  {
  cat("Choose a shape to calculate the area:\n")
  cat("1. Circle\n")
  cat("2. Rectangle\n")
  cat("3. Triangle\n")
  shape <- as.integer(readline(prompt = "Enter the number corresponding to the shape: "))
    if (shape == 1)
  {
    radius <- as.numeric(readline(prompt = "Enter the radius of the circle: "))
    area <- area_circle(radius)
```

```
    cat("The area of the circle is:", area, "\n")
  } else if (shape == 2) {
    length <- as.numeric(readline(prompt = "Enter the length of the rectangle: "))
    width <- as.numeric(readline(prompt = "Enter the width of the rectangle: "))
    area <- area_rectangle(length, width)
    cat("The area of the rectangle is:", area, "\n")
    } else if (shape == 3){
        base <- as.numeric(readline(prompt = "Enter the base of the triangle: "))
        height <- as.numeric(readline(prompt = "Enter the height of the triangle: "))
        area <- area_triangle(base, height)
        cat("The area of the triangle is:", area, "\n")
     }    else {
    cat("Invalid choice. Please run the program again and select a valid option.\n")
     }
  }
calculate_area()
```

Output:

Choose a shape to calculate the area:

1. Circle

2. Rectangle

3. Triangle

Enter the number corresponding to the shape: 3

Enter the base of the triangle: 5

Enter the height of the triangle: 10

The area of the triangle is: 25

Result:

Thus the R Program to find the area of different shape using user defined functions has been executed successfully.

EX.NO:3b	RECURSIVE FUNCTIONS

Aim:

To write R Program to find factorial of a number using recursive functions.

Program:

```
factorial_recursive <- function(n)
{
  if (n == 0)
  {
    return(1)
  } else {
    return(n * factorial_recursive(n - 1))
  }
}

n <- as.integer(readline(prompt = "Enter a number: "))
result <- factorial_recursive(n)
print(paste("Factorial of", n, "is:", result))
```

Output

```
Enter a number: 5
[1] "Factorial of 5 is: 120"
```

Result:

Thus the R Program to find factorial of a number using recursive functions has been executed successfully.

EX.NO:4a	VECTORS

Aim:

To write R Program to Count Even and Odd Numbers using vector.

Program:

```
numbers <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
even_count <- 0
odd_count <- 0
for (i in numbers) {
  if (i %% 2 == 0) {
    even_count <- even_count + 1
  } else {
    odd_count <- odd_count + 1
  }
}
print(paste("Number of even numbers:", even_count))
print(paste("Number of odd numbers:", odd_count))
```

Output:

[1] "Number of even numbers: 5"

[1] "Number of odd numbers: 5"

Result:

Thus the R Program to Count Even and Odd Numbers using vector has been executed successfully.

EX.NO:4b	VECTORS TO DATAFRAME

Aim:

To write R Program to Prepare Student Mark Sheet using vector convert into Data frame

Program:

```
name = c('Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael', 'Matthew',
'Laura', 'Kevin', 'Jonas')
score1 = c(70, 90, 76, 52, 92, 70, 46, 83, 89, 90)
score2= c(85, 75, 82, 33, 72, 83, 51, 61, 72, 81)
result = c('Pass', ' Pass ', ' Pass ', 'Fail', ' Pass ', ' Pass ', ' Pass ', ' Pass ', ' Pass ', ' Pass ')
print("Original data frame:")
print(name)
print(score1)
print(score2)
print(result)
df = data.frame(name, score1, score2, result)
print(df)
```

Output:

```
[1] "Original data frame:"
[1] "Anastasia" "Dima"     "Katherine" "James"     "Emily"     "Michael"
[7] "Matthew"   "Laura"     "Kevin"     "Jonas"
[1] 70 90 76 52 92 70 46 83 89 90
[1] 85 75 82 33 72 83 51 61 72 81
[1] "Pass"   " Pass " " Pass " "Fail"   " Pass " " Pass " " Pass " " Pass "
[9] " Pass " " Pass "
```

	name	score1	score2	result
1	Anastasia	70	85	Pass
2	Dima	90	75	Pass
3	Katherine	76	82	Pass
4	James	52	33	Fail
5	Emily	92	72	Pass
6	Michael	70	83	Pass
7	Matthew	46	51	Pass
8	Laura	83	61	Pass
9	Kevin	89	72	Pass
10	Jonas	90	81	Pass

Result:

Thus the R Program to Prepare Student Mark Sheet using vector convert into Data frame has been executed successfully.

EX.NO:4c	LIST

Aim:

To write R Program to Check if an Element exists in a List

Program:

```
my_list <- list(1, "Hello", TRUE, c(1, 2, 3))  # A mixed list
x <- readline(prompt = "Enter element to check: ")  # User input
if (suppressWarnings(!is.na(as.numeric(x)))) {
  x <- as.numeric(x)
} else if (tolower(x) == "true") {
  x <- TRUE
} else if (tolower(x) == "false") {
  x <- FALSE
}
if (x %in% unlist(my_list)) {
  print(paste(x, "is present in the list"))
} else {
  print(paste(x, "is not present in the list"))
}
```

Output:

```
Enter element to check: Hello
[1] "Hello is present in the list"
```

Result:

Thus the R Program to check if an Element Exists in a List has been executed successfully

EX.NO:5	ARRAYS

Aim:

To write R Program to perform Matrix Operation such as Addition, Subtraction, Multiplication, Transpose and Inverse.

Program:

```
A<- matrix(c(1, 2, 3, 4, 5, 6, 7, 8, 9), nrow = 3, ncol = 3)
B <- matrix(c(9, 8, 7, 6, 5, 4, 3, 2, 1), nrow = 3, ncol = 3)
 matrix_sum<-A + B
cat("MatrixAddition:\n")
print(matrix_sum)
matrix_diff<-A - B
cat("MatrixSubtraction:\n")
print(matrix_diff)
matrix_product<-A%*%t(B)
cat("MatrixMultiplication:\n")
print(matrix_product)

matrix_tra<-t(A)
cat("Transpose:\n")
print(matrix_tra)

 if (det(A) != 0) {
 matrix_inv <- solve(A)
 cat("Matrix Inverse:\n")
```

```
  print(matrix_inv)
} else {
  cat("Matrix A is singular and cannot be inverted.\n")
}
```

Output:

```
MatrixAddition:
     [,1] [,2] [,3]
[1,]   10   10   10
[2,]   10   10   10
[3,]   10   10   10
MatrixSubtraction:
     [,1] [,2] [,3]
[1,]   -8   -2    4
[2,]   -6    0    6
[3,]   -4    2    8
```

```
MatrixMultiplication:
     [,1] [,2] [,3]
[1,]   54   42   30
[2,]   72   57   42
[3,]   90   72   54
Transpose:
     [,1] [,2] [,3]
[1,]    1    2    3
```

```
[2,]    4    5    6

[3,]    7    8    9
```

Matrix A is singular and cannot be inverted.

Result:

Thus the R Program to perform Matrix Operation such as Addition, Subtraction, Multiplication, Transpose and Inverse has been executed successfully

EX.NO:6 DATA INTERFACE: EXCEL FILE, CSV FILES

Aim

Write an R Program to Read Excel file and CSV file using Data interface.

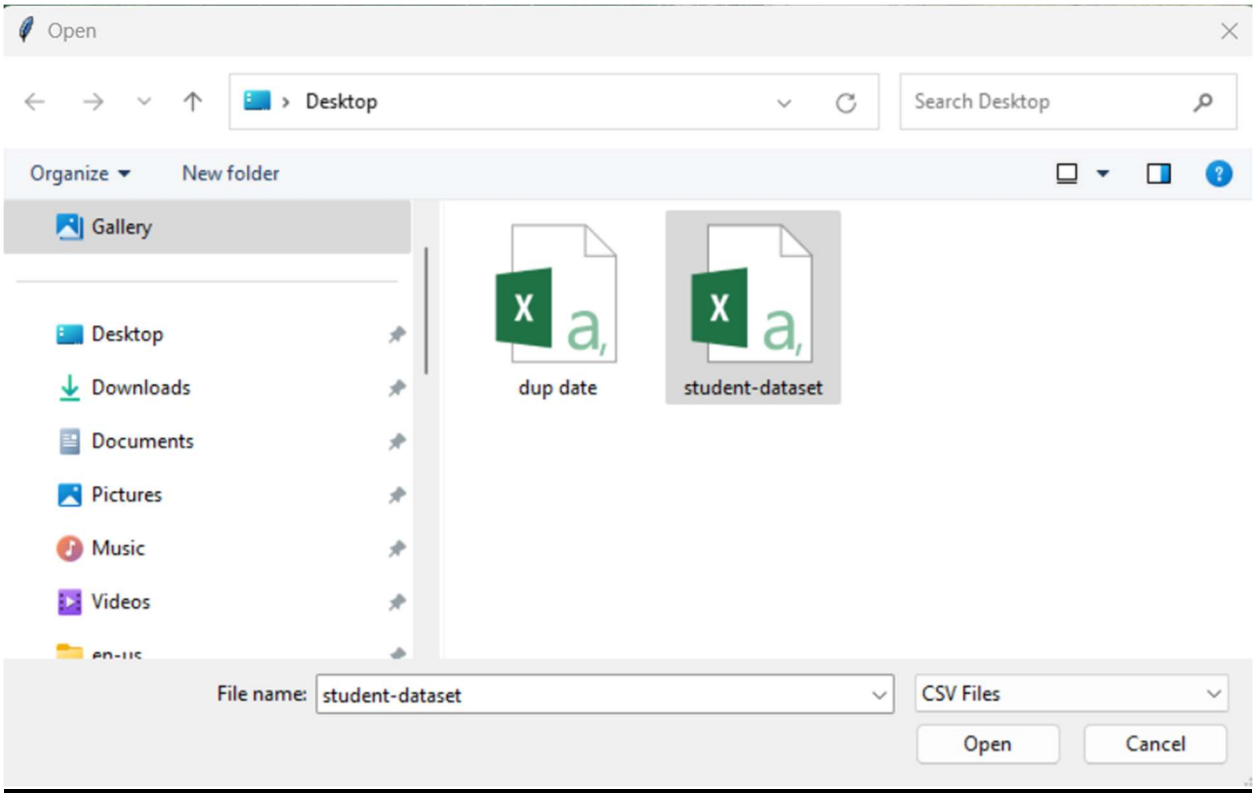
Program

```
library(readxl)
library(readr)
library(tcltk)
library(dplyr)
# Function to read CSV file
read_csv_file <- function(filepath) {
  cat("Reading CSV file:", filepath, "\n")
  df <- read_csv(filepath)
  return(df)
}
# Function to read Excel file
read_excel_file <- function(filepath)
 {
        cat("Reading Excel file:", filepath, "\n")
         sheets <- excel_sheets(filepath)
        cat("Available sheets:", paste(sheets, collapse = ", "), "\n")
        df <- read_excel(filepath, sheet = sheets[1])  # Default: read first sheet
        return(df)
}
```

```
# File selection dialog
select_file <- function()
{
  filepath <- tclvalue(tkgetOpenFile(filetypes = "{{CSV Files} {.csv}} {{Excel Files} {.xls
.xlsx}}"))
  return(filepath)
}
# Main interface
main <- function()
 {
  filepath <- select_file()
  if (filepath == "")
  {
    cat("No file selected. Exiting.\n")
    return(NULL)
  }
  if (grepl("\\.csv$", filepath, ignore.case = TRUE))
{
    data <- read_csv_file(filepath)
  } else if (grepl("\\.xls[x]?$", filepath, ignore.case = TRUE))
  {
    data <- read_excel_file(filepath)
  } else {
    cat("Unsupported file format.\n")
    return(NULL)
```

```
  }
  # Preview the data
  cat("Preview of data:\n")
  print(head(data))
    return(data)
}
data_loaded <- main()
```

Output



id	name	nationality	gender	age	english.grade	math.grade	sciences.grade	language.grade
0	Kiana Lor	China	F	22	3.5	3.7	3.1	1
1	Joshua Lonaker	United States of America	M	22	2.9	3.2	3.6	5
2	Dakota Blanco	United States of America	F	22	3.9	3.8	3.2	5
3	Natasha Yarusso	United States of America	F	20	3.3	2.8	3.2	5
4	Brooke Cazares	Brazil	F	21	3.7	2.6	3.4	1
5	Rochelle Johnson	United States of America	F	21	3.4	3.1	3.7	5
6	Joey Abreu	China	M	22	3.7	3.9	3.6	2

Result

Thus the R program Read Excel file and CSV file using Data interface using Data interface has been executed successfully.

EX.NO:7 GRAPHS: UNIVARIATE, BIVARIATE AND MULTIVARIATE

Aim:

Write a R program for any visual representation of an object with creating graphs using graphic functions:

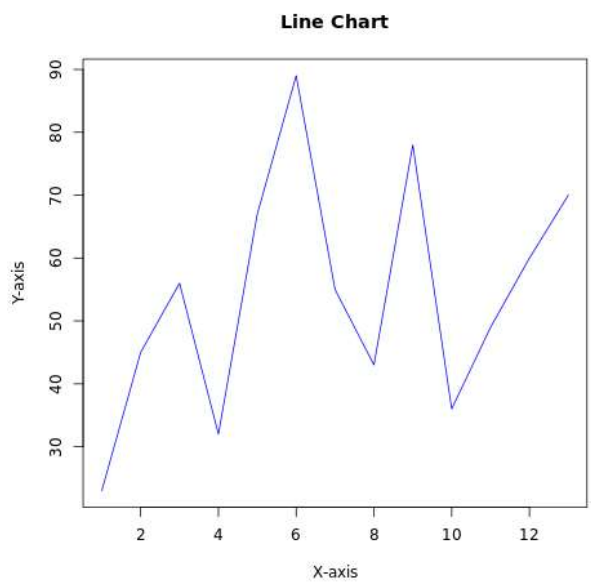
Plot(), Hist(), Linechart(), Pie(), Boxplot(), Scatterplots()

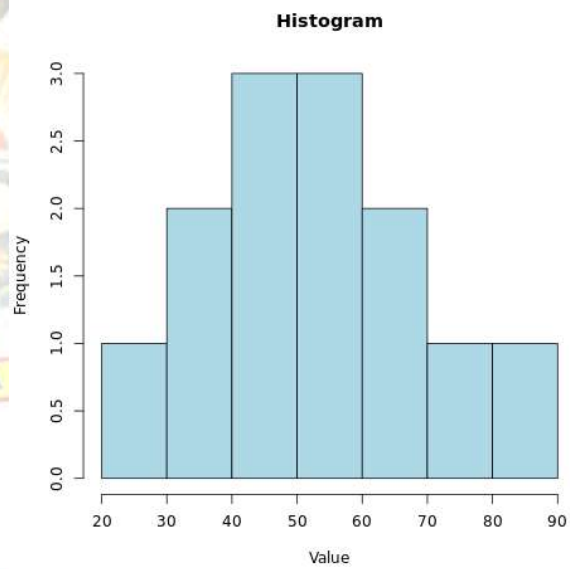
Program:

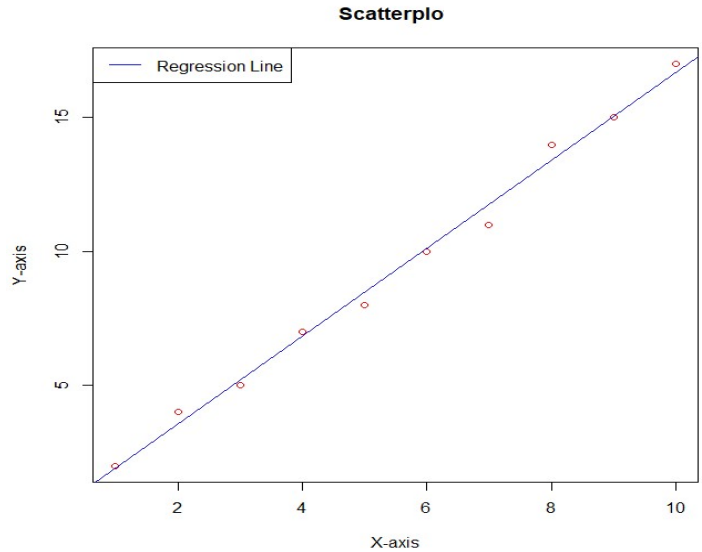
```
data <- c(23, 45, 56, 32, 67, 89, 55, 43, 78, 36, 49, 60, 70)

# Line chart
plot(data, type = "l", col = "blue", xlab = "X-axis", ylab = "Y-axis", main = "Line
Chart")

# Histogram
hist(data, col = "lightblue", xlab = "Value", ylab = "Frequency", main = "Histogram")

# Pie chart
pie_data <- c(20, 30, 40, 10)
pie(pie_data, labels = c("A", "B", "C", "D"), col = rainbow(length(pie_data)), main =
"Pie
Chart")
# Boxplot
boxplot(data, col = "lightgreen", xlab = "Value", main = "Box Plot")
# Scatterplot
```

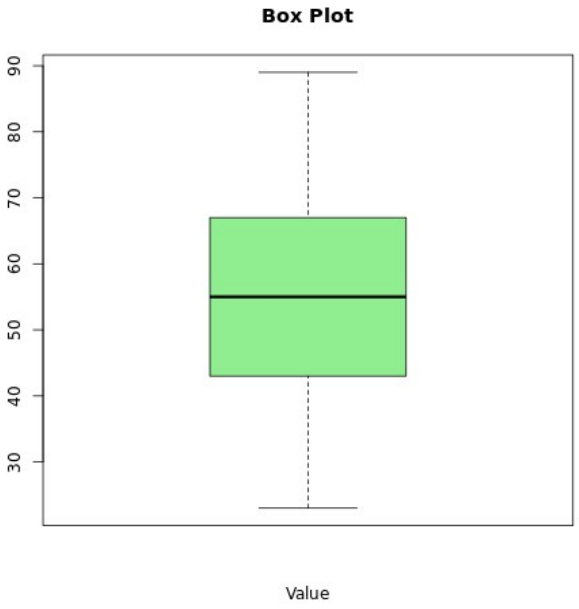
```
x <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
y <- c(2, 4, 5, 7, 8, 10, 11, 14, 15, 17)
plot(x, y, col = "red", xlab = "X-axis", ylab = "Y-axis", main = "Scatterplot")
#Add a regression line to the scatterplot
abline(lm(y ~ x), col = "blue")
#Create a legend for the scatterplot
legend("topleft", legend = "Regression Line", col = "blue", lty = 1)
```

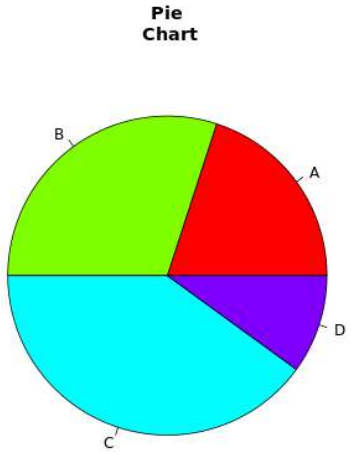
Output:











Result:

Thus the R program for any visual representation of an object with creating graphs using graphic functions has been executed successfully.

EX.NO:8 PLOT FUNCTION: CORRELATION PLOTS, SURVIVAL PLOTS AND MOSAIC PLOTS

Aim:

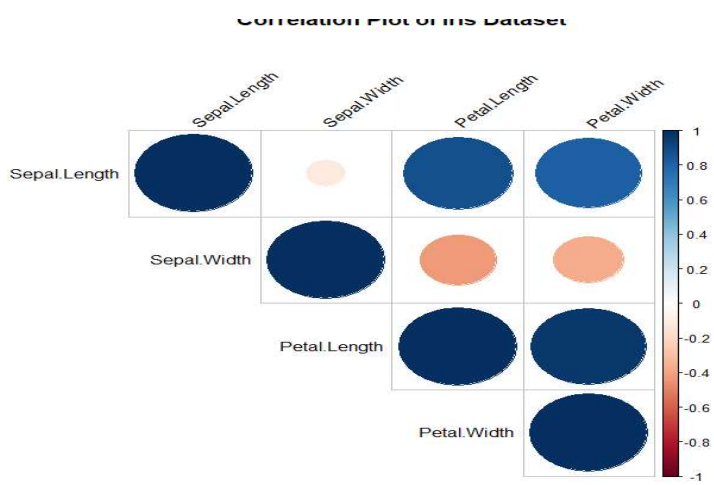
Write a R program for any plot function: correlation plots, survival plots and mosaic plots.

Program:

Correlation Plot

```
install.packages("corrplot")

library(corrplot)

data(iris)

cor_matrix <- cor(iris[,1:4])

corrplot(cor_matrix, method = "circle", type = "upper", tl.col = "black", tl.srt = 45, title = "Correlation Plot of Iris Dataset")
```

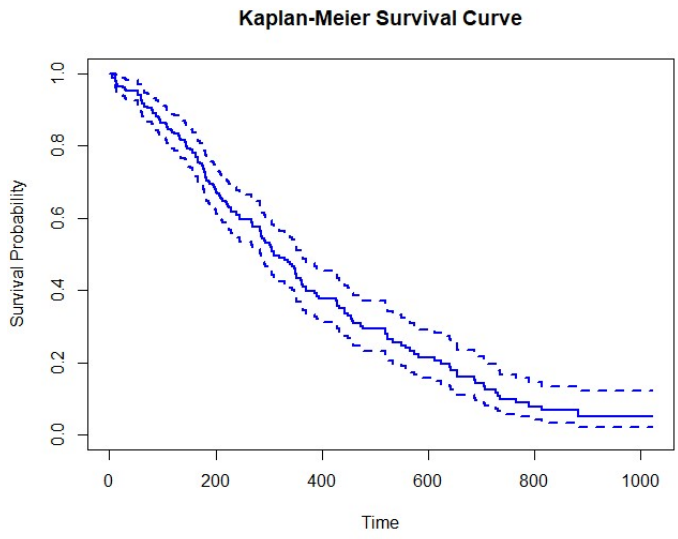
Output:



Survival Plot

```
install.packages("survival")
library(survival)
data(lung)  # lung cancer dataset
head(lung)
surv_obj <- Surv(lung$time, lung$status)
fit <- survfit(surv_obj ~ 1, data = lung)
    plot(fit, main = "Kaplan-Meier Survival Curve",
    xlab = "Time",
    ylab = "Survival Probability",
    col = "blue",
    lwd = 2)
```

Output:

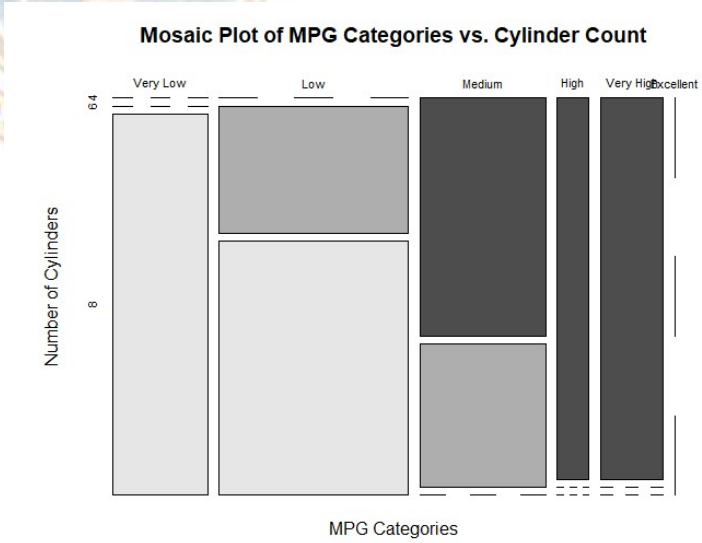


Mosaic plot

```
data(mtcars)
mtcars$mpg_cat <- cut(mtcars$mpg,
                      breaks = c(0, 15, 20, 25, 30, 35, 40),
                      labels = c("Very Low", "Low", "Medium", "High", "Very High",
"Excellent"))
mtcars$cyl_factor <- factor(mtcars$cyl)
contingency_table <- table(mtcars$mpg_cat, mtcars$cyl_factor)
print(contingency_table)
mosaicplot(contingency_table,
        main = "Mosaic Plot of MPG Categories vs. Cylinder Count",
        xlab = "MPG Categories",
        ylab = "Number of Cylinders",
        color = TRUE)  # color=TRUE adds color to the tiles
```

Output:

	4	6	8
Very Low	0	0	6
Low	0	4	8
Medium	5	3	0
High	2	0	0
Very High	4	0	0
Excellent	0	0	0



Result:

Thus the R program for any plot function: correlation plots, survival plots and mosaic plots has been executed successfully.

EX.NO:9	MEASURE OF CENTRAL TENDENCY- MEAN, MEDIAN AND MODE

Aim:

Write a R Program to perform measure of central tendency: mean, median and mode.

Program:

```
data <- c(1, 2, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9)
mean_value <- mean(data)
cat("Mean:", mean_value, "\n")
median_value <- median(data)
cat("Median:", median_value, "\n")
mode_value <- function(x) {
  uniq_x <- unique(x)
  uniq_x[which.max(tabulate(match(x, uniq_x)))]
}
mode_result <- mode_value(data)
cat("Mode:", mode_result, "\n")
```

Output:

Mean: 4.75

Median: 5

Mode: 5

Result:

Thus the R perform measure of central tendency, mean, median and mode has been executed successfully.

EX.NO:10	STANDARD DEVIATION

Aim:

Write a R Program to Perform standard deviation.

Program:

```
data <- c(5, 10, 15, 20, 25, 30, 35, 40, 45, 50)
std_dev <- sd(data)
print(paste("Standard Deviation:", std_dev))
```

Output

```
[1]    "Standard Deviation: 15.1382517704875"
```

Result:

Thus the R program using standard deviation is Executed Successfully.

EX.NO:11	CORRELATIONS AND COVARIANCE

Aim:

Write a R Program to Perform correlations and covariance.

Program:

```
x <- c(1, 6,5,10,3,2,4,9,7,8)
y <- c(3,5,8,4,7,10,2,1,6,9)

# Calculate Covariance
covariance <- cov(x, y)
print(paste("Covariance between x and y:", covariance))

# Calculate Correlation
correlation <- cor(x, y)
print(paste("Correlation between x and y:", correlation))
```

Output:

```
[1] "Covariance between x and y: -1.94444444444444"
[1] "Correlation between x and y: -0.212121212121212"
```

Result:

Thus the R program correlations and covariance is has been executed successfully.

EX.NO:12a **SIMPLE LINEAR REGRESSION**

Aim:

Write R Program to Perform Simple Linear Regression

Program:

```
x <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)  #Independent Variable

y <- c(2, 4, 5, 4, 5, 7, 8, 9, 10, 12) #Dependent Variable

model <- lm(y ~ x)

print(summary(model))

plot(x, y, main = "Simple Linear Regression", xlab = "X (Predictor)", ylab = "Y
(Response)", pch = 16)

abline(model, col = "blue")
```

Output:

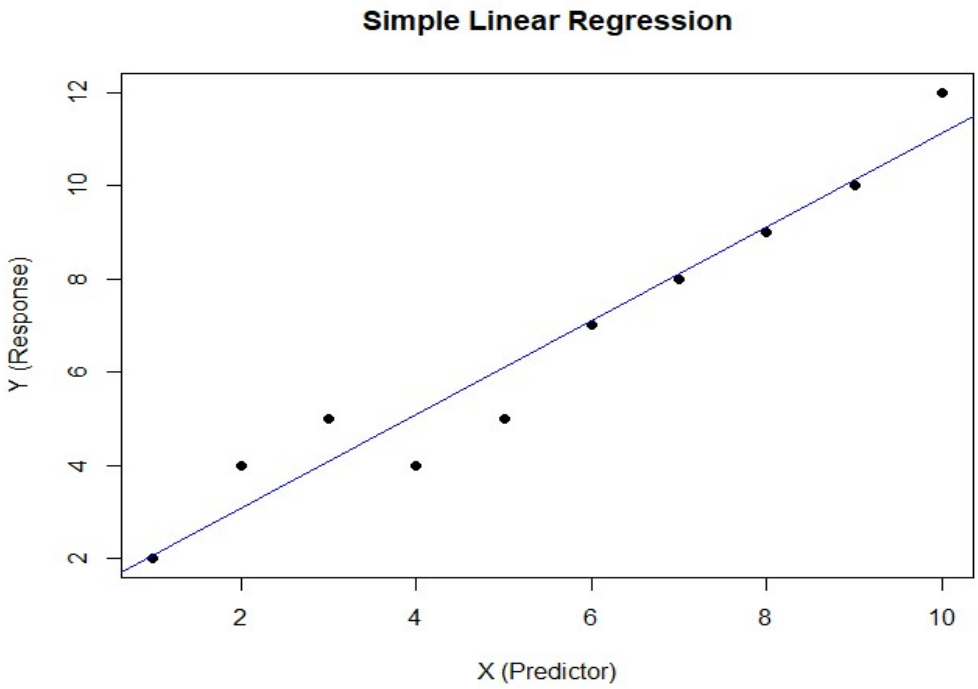
```
Call:

lm(formula = y ~ x)


Residuals:

    Min      1Q  Median      3Q     Max

-1.0970 -0.1197 -0.1061  0.6364  0.9212


Coefficients:

            Estimate Std. Error t value Pr(>|t|)

(Intercept)  1.06667    0.53447   1.996    0.081 .

x            1.00606    0.08614  11.680 2.63e-06 ***

---
```

Signif. codes: 0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.7824 on 8 degrees of freedom

Multiple R-squared: 0.9446, Adjusted R-squared: 0.9377

F-statistic: 136.4 on 1 and 8 DF, p-value: 2.634e-06



Result:

Thus the R program Simple Linear Regression has been executed successfully.

EX.NO:12b **MULTIPLE LINEAR REGRESSION**

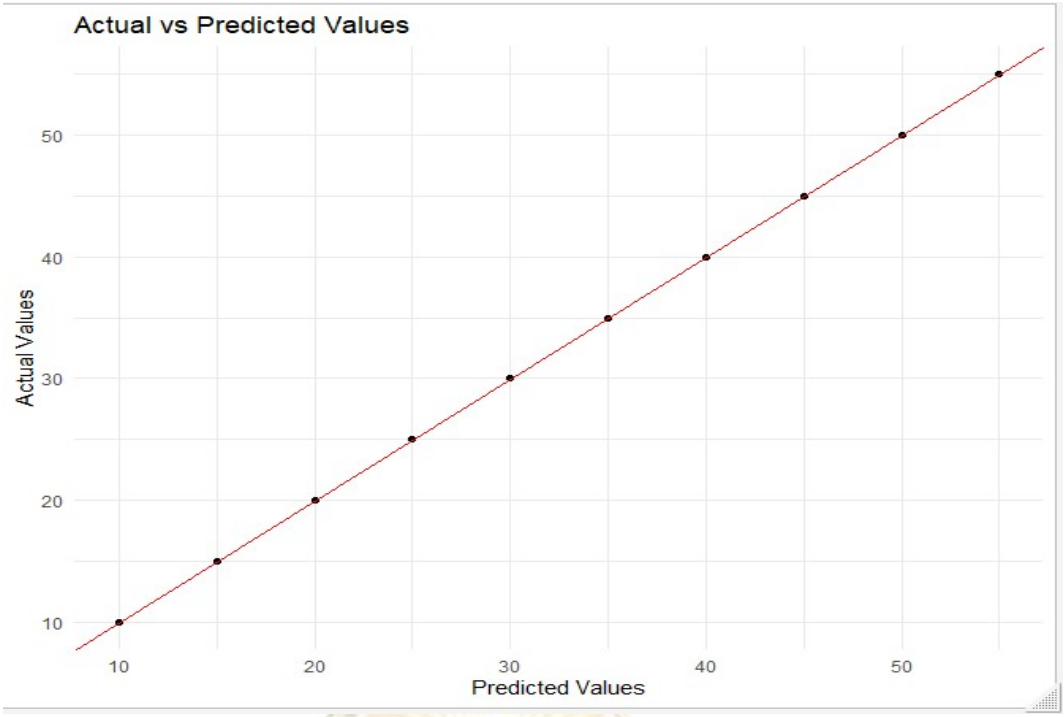
Aim:

Write R Program to Perform Multiple Linear Regression

Program:

```
set.seed(42)  # For reproducibility

x1 <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)  # Independent variable1

x2 <- c(5, 4, 6, 8, 7, 8, 9, 9, 10, 11) # Independent variable1

y  <- c(10, 15, 20, 25, 30, 35, 40, 45, 50, 55)  # Dependent variable

data <- data.frame(x1 = x1, x2 = x2, y = y)

model <- lm(y ~ x1 + x2, data = data)

summary(model)

library(ggplot2)

data$predicted_y <- predict(model)

ggplot(data, aes(x = predicted_y, y = y)) +  geom_point() +  geom_abline(slope = 1,
intercept = 0, color = "red") +  labs(title = "Actual vs Predicted Values", x = "Predicted
Values", y = "Actual Values") +  theme_minimal()
```

Output:



Result:

Thus the R program Multiple Linear Regression has been executed successfully.